Pointers: Addresses and Arrays

Matteo Fasiolo

Previously

- Array elements occupy contiguous memory.
- An array function normally receives the array and its length.
- A function call such as `add_one(data, length)` can modify the elements of `data`.

Now we clarify what is really happening in the last point.

Consider the function:

```
void add_one(double values[], int length) {
    for (int i = 0; i < length; i = i + 1) {
        values[i] = values[i] + 1.0;
    }
}
```

We can call it by

```
double data[] = {1.4, 0.1, /* rest omitted */};

add_one(data, length);
```

which changes the elements of `data`.

C does not copy the entire array.

Instead, the call passes the **address** of `data[0]`.

Physical memory and a program's addresses

Physical RAM stores the code and data used by running programs.



A program normally works through a **virtual address space** supplied by the operating system.

It does not need to manage individual RAM chips.

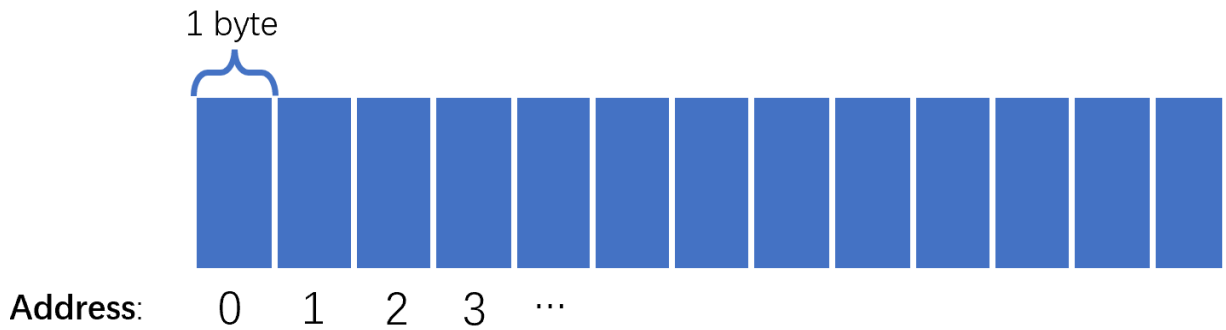
Virtual address space is an **abstraction** of physical memory.

Recall the distinction between **low- and high-level languages**.

Memory is byte-addressed

Conceptually, virtual memory is a sequence of bytes.

Each byte has an address.

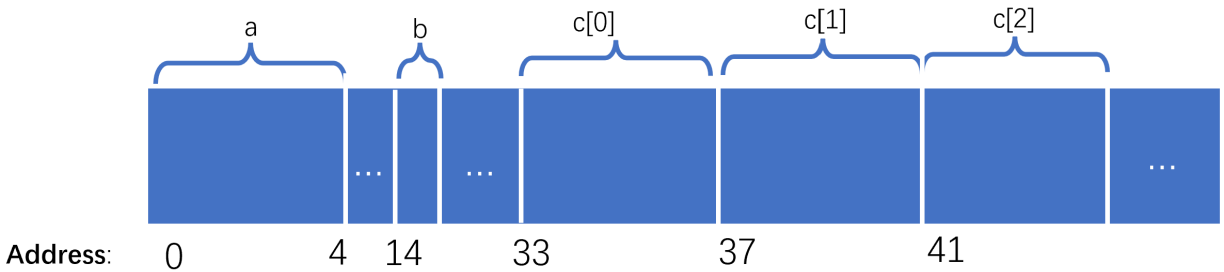


An address identifies a location.

The content stored there is a separate idea.

Variables may occupy several bytes

Variables of different types may require different numbers of bytes.



- The exact size of a type is platform-dependent.
- Array elements have one type and occupy consecutive regions.

A variable has a value and an address

```
int x = 10;
```

Two different expressions:

- `x` gives the stored integer value, 10;
- `&x` gives the address where `x` is stored.

The type of `x` is `int`.

The type of `&x` is "pointer to `int`", written `int *`.

Pointer

A **pointer** is a variable whose value is an address:

```
int x = 10;
int *p = &x;
```

Read the declaration as:
p is a pointer to an int.

The pointer type describes the type of value that may be accessed through it.

For example

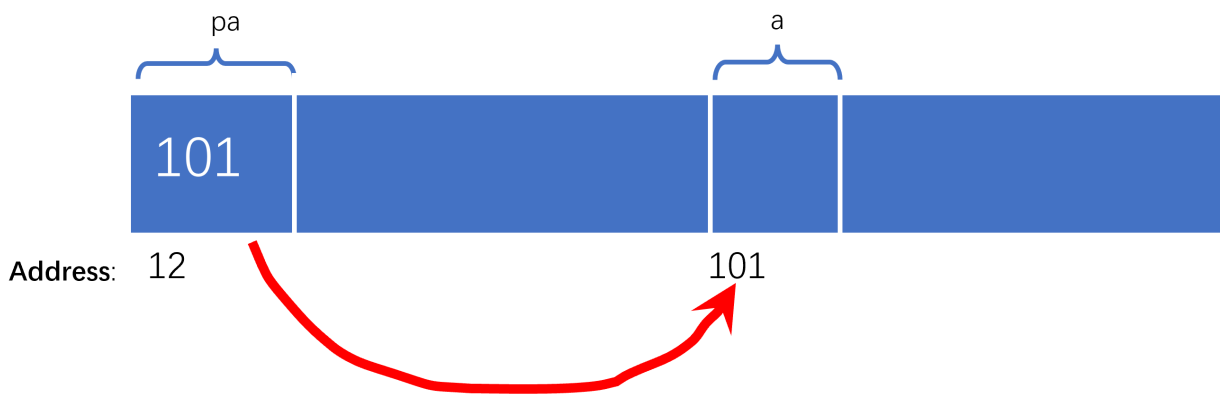
```
double x = 3.3;
double *p = &x;
```

is a pointer to a double.

Memory diagram of a pointer

```
int a = 0;
int *pa = &a;
```

- pa stores the address of `int` variable a.
- pa itself is also a variable and is stored in the memory.



The pointer pa is itself a variable, stored separately from a.

Dereferencing a pointer

Consider the code:

```
int x = 10;
int *p = &x;

printf("%d\n", *p);  // prints 10
```

Here `*` is used in two ways:

- in `int *p = &x;` it declared that `p` is pointer
- in `printf("%d\n", *p);` it **dereferences** the pointer.

Dereferencing means accessing the value that `p` is pointing to.

In this code:

```
int x = 3;
int *p = &x;
printf("%d\n", *p);
```

there are four expressions to distinguish:

Expression	Meaning	Type
`x`	value stored in `x`	`int`
`&x`	address of `x`	`int *`
`p`	address stored in `p`	`int *`
`*p`	value at the address stored in `p`	`int`

When `p == &x`, the expressions `x` and `*p` refer to the same integer variable.

Consider

```
int x = 10;
int *p = &x;
```

We can write using a pointer:

```
*p = 25;
```

After the assignment:

```
x  == 25
*p == 25
```

Similarly after

```
x = 3;
```

both `x` and `*p` are equal to 3.

There is only one integer: `x` and `*p` are two ways to access it.

Printing

`*p` can be used in the same ways as `x`. For example printing:

```
printf("%d \n", *p);
```

Printing `p` is different, we are printing an address.

Use `%p` and convert the pointer argument to `void *`:

```
printf("%p\n", (void *)p);
printf("%p\n", (void *)&x);
// NOTE: I won't ask you about (void *)
```

The displayed address:

- is intended for inspection and debugging;
- may differ between program runs;

Switch to VS Code

Open `pointer_basics.c` (PART I)

The debugger confirmed:

- `p` and `&x` contained the same address;
- `x` and `*p` accessed the same stored integer;
- changing `*p` changed `x`;
- changing `x` changed `*p`;

Passing pointers to functions

```
void increment(int *p) {
    *p = *p + 1;
}

int main(void) {
    int x = 10;
    increment(&x);
}
```

The call copies the address value `&x` into the local parameter `p`.

Dereferencing that copied address accesses the caller's original `x`.

Again:

- `int *p` is declaring that `p` is a pointer
- in `*p = *p + 1;` we are dereferencing the pointer

Suppose we want to swap two variables

```
int a = 3;
int b = 7;
swap(&a, &b);
```

We are passing to the swapping function two addresses.

The function is:

```
void swap(int *left, int *right) {
    int temporary = *left;
    *left = *right;
    *right = temporary;
}
```

After the call, `a == 7` and `b == 3`.

A pointer must contain a valid address

An uninitialised pointer is not ready to use:

```
int *p;         // not ready to use
```

Do not print it, dereference it or guess where it points.

Initialise a pointer with a valid address or with `NULL`:

```
int *p = NULL;
```

`NULL` means “does not point anywhere”.

It must not be dereferenced.

You can check before dereferencing

```
if (p != NULL) {
    printf("%d\n", *p);
}
```

Dereferencing an invalid or null pointer has undefined behaviour.

Note: **you** need to check, by using code as above.

The compiler will not check:

```
int *p = NULL;
printf("%d\n", *p);
```

may compile and run, but might crash (more on this later)!

Live demonstration 2

Switch to VS Code

Open `pointer_basics.c` (PART II)

Pointers and arrays

Consider

```
int values[] = {2, 3, 4};
int *p = values;
```

What does the second line do?

It is equivalent to

```
int *p = &values[0];
```

Now `p` contains the address of `values[0]`.

If you have a function such as

```
void add_one(double *values, int length)
```

and you call it by doing

```
double array[3] = {1.0, 2.0, 3.0};

add_one(array, 3);
```

Then

```
add_one(array, 3);
```

is equivalent to

```
add_one(&array[0], 3);
```

and `values` contains the address of `array[0]`;

In most expressions, an array expression such as

```
double *x = array;
\\ or
add_one(array, 3);
```

is automatically converted to a pointer to its first element.

So, you can write array parameters two equivalent ways.

Using arrays

```
void add_one(double values[], int length)
```

or pointers

```
void add_one(double *values, int length)
```

In both cases the function receives a copied address.

This is why it can access the caller's elements.

Pointer indexing

If p points to the first element of an array, pointer indexing works:

```
int values[] = {2, 3, 4};
int *p = values;

printf("%d\n", p[2]);  // prints 4
```

So pointers can be used similarly to arrays, but:

- they have different types and different storage.
- they are not universally interchangeable.

Note that the third line below does not make sense:

```
int x = 7;
int *p = &x;
printf("%d\n", p[2]);
```

Consider

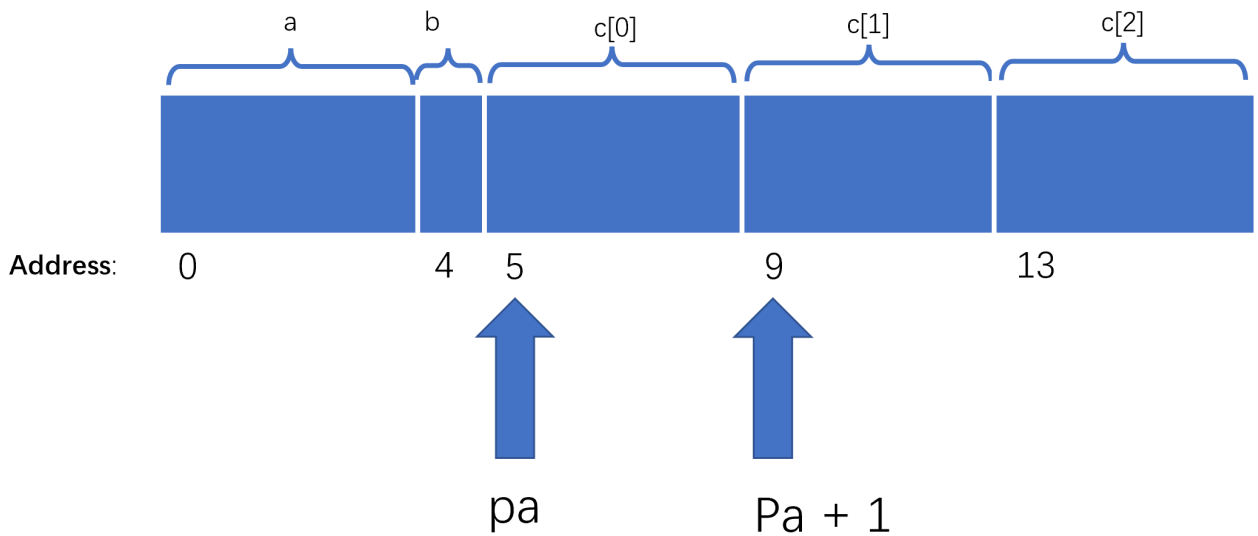
```
int values[] = {2, 3, 4};
int *pa = values;
```

Then `pa[i]` is equivalent to `*(pa + i)`.

The code `pa + i` moves to the pointer of the i-th array element.

The change in the address depends on the pointed-to type.

If `pa` points to an `int`, then `pa + 1` points to the next `int`.



As for arrays, we still need to check the bounds.

Pointer notation does not add bounds checking:

```c
int values[3] = {2, 3, 4};
int *p = values;

printf("%d\n", *(p + 3));  // out of bounds
```

Valid offsets are 0, 1 and 2.

Switch to VS Code

Open `pointer_arrays.c`

The debugger confirmed:

- the array expression located the first element;
- pointer indexing and array indexing accessed the same elements;
- `p + i` moved in units of the pointed-to type;
- none of these operations supplied automatic bounds checking.

Check your understanding

```c
int x = 5;
int *p = &x;
*p = 8;
```

1. What are the types and meanings of `x`, `&x`, `p` and `*p`?
2. What changes after `*p = 8`?
3. What value is copied into `p`, and why can `*p` change `x`?
4. What does the following code print:

```c
int a[4] = {4, 7, 8, 1};
int *p = a;

printf("%d \n", *(p + 2));
```

?

Conclusion

1. A pointer stores an address and has a pointed-to type.
2. `&x` obtains the address of `x`; `*p` accesses the target of `p`.
3. Pointer parameters receive copied addresses and can access the calling function's data.
4. Arrays convert to pointers to their first elements in most expressions.
5. `p[i]` and `*(p+i)` are equivalent (for valid indices).
6. Only dereference a pointer known to point to a valid value.